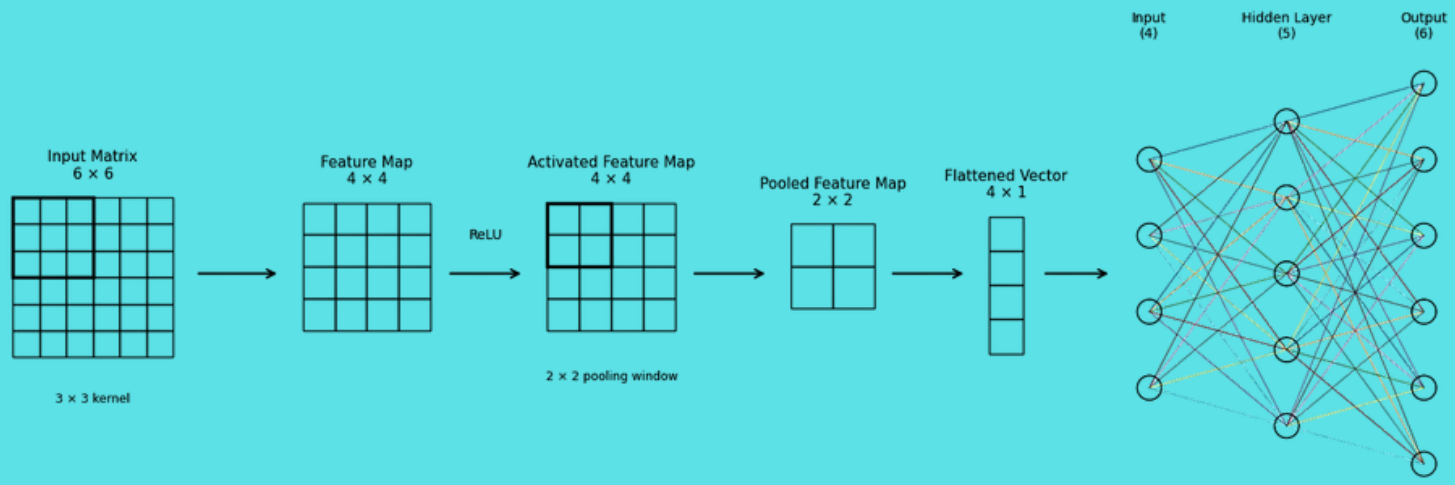
Modules for
Machine and Deep Learning

Module 3:
Convolutional Neural Network (CNN) and a Surrogate Model for FEM – With PyTorch



Assoc. Prof. Ir. Dr. Airil Yasreen Mohd Yassin
School of Energy, Geoscience, Infrastructure and Society, Heriot-Watt University Malaysia

Assoc. Prof. Ts. Dr. Ahmad Razin Zainal Abidin
Faculty of Civil Engineering, Universiti Teknologi Malaysia

Dr. Halinawati Hirol
Malaysia-Japan International Institute of Technology (MJIIT), Universiti Teknologi Malaysia

Dr. Mokhtazul Haizad Mokhtaram
Faculty of Engineering and Life Sciences, Universiti Selangor

Dr. Mohd Al-Akhbar Mohd Noor
Faculty of Engineering and Life Sciences, Universiti Selangor

Table of Contents

Modules for Machine and Deep Learning (Module 3: CNN and a surrogate model for FEM with PyTorch)

1. Introduction

In Modules 1 and 2, we focused on the first principles and fundamentals of Neural Networks, or more specifically, Feedforward Neural Networks (FFNNs). In this Module 3, we will expand our horizon by discussing another variation of NN, called the Convolutional Neural Network (CNN).

CNNs are widely used for image (and video) and audio processing and have found prominent roles in industries such as medical and entertainment. However, CNNs are also used in many other fields, including engineering.

In structural engineering, for example, CNNs have been used alongside traditional numerical methods such as the Finite Element Method (FEM) (Bolandi, et. al. (2022)), Finite Volume Method (FVM) (Zhang et. al. (2023)) and Finite Difference Method (FDM) (Celedoni et.al. (2025)). In Structural Health Monitoring (SHM), CNNs have also been utilized for inverse problems, where measured structural responses such as natural frequencies and mode shapes can be used to identify structural damage and changes in stiffness (Zhong, et.a. (2020)).

2. CNN Architecture

The distinguishing feature of CNN is the additional layer before the FFNN, as shown in Figure 1, called the convolutional layer. In this additional layer, there are processes of convolving the input with a kernel (or filter), applying a nonlinear activation function to the resulting feature map, and pooling the activated feature map to reduce its spatial size. This process can be repeated, resulting in a few blocks or stacks of convolutional layers, before the final pooled feature map is flattened as input to the FFNN.

Another distinguishing feature of CNN (which governs the choice of our mathematical pedagogy) is that it works with matrix input for a single-channel input and tensor input for multi-channel inputs (e.g., RGB images). We opt for the former due to the introductory nature of our module.

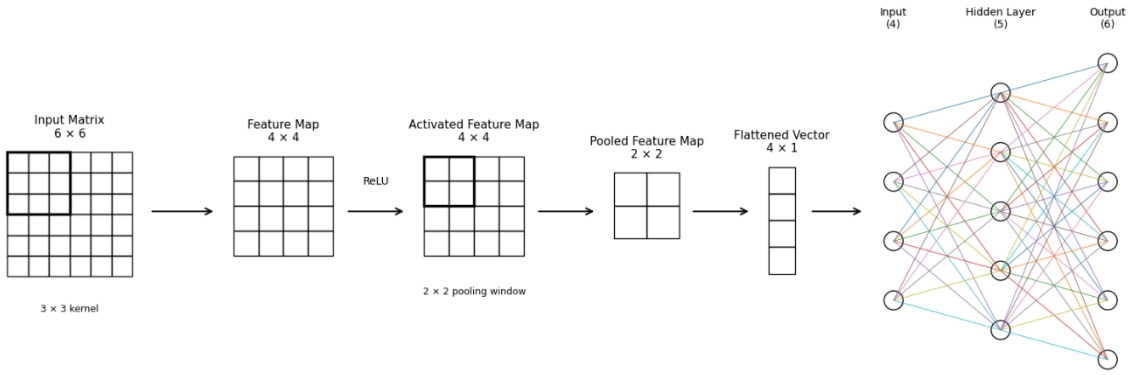


Figure 1: CNN architecture used in this module

3. How we are going to proceed with the module (pedagogical strategy)

The commonality between CNN for image processing and engineering analysis is the availability of input data in the form of a matrix. In CNN, this input matrix can represent the pixels of a black-and-white image. For a coloured image, there will be three "layers", or more appropriately termed

as channels, of such input, which form an input tensor. For example, an RGB image consists of three channels representing Red, Green and Blue, respectively. As mentioned, however, for our introductory discussion, we will stick with a single input matrix.

For engineering problems, such a single matrix input can be the coefficient matrix of a discretized governing equation, which can typically be given as:

$$[\bar{K}]\{U\} = \{\bar{F}\} \quad (1)$$

where $[\bar{K}]$ is the coefficient matrix (or, in FEM, the stiffness matrix) hence the input matrix, $\{U\}$ is the degree-of-freedom (dof) vector and $\{\bar{F}\}$ is the load vector. The availability of such matrix-form data provides an opportunity for the application of CNN in engineering problems.

Now we have a choice on how to proceed with our discussion, either to work with an input matrix from an image or with a generated $[\bar{K}]$ from the hypothetical engineering equation in Eq. (1). To uphold the philosophy and pedagogical strategy of Modules 1 and 2, we will opt for the latter because we believe that, if the "maths" is understood, subsequent extension and application towards more "industrial" practices will be more straightforward. This also means we will work with a regression problem instead of classification.

4. The "Maths" Behind CNN: Symbolic Hand-Calculation of the Convolutional Processing

Now that we have established the architecture of the convolutional layer (Figure 1) and where we obtain the input matrix $[\bar{K}]$ (that is, from Eq. (1)), we can describe the mathematical processes involved as the input passes through the convolutional layer before getting into the FFNN as input. We will do this symbolically first, before we get to the stage of PyTorch implementation.

The workflow of the convolutional processing is given in Figure 2.

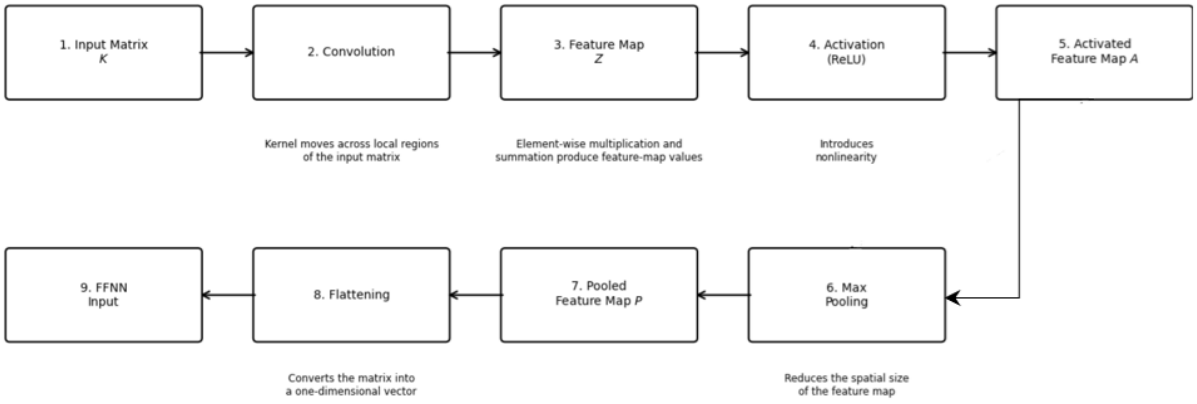


Figure 2: Workflow of the convolutional processing

Now, specifically for this module, our raw input matrix is $[\bar{K}]$, which is generated independently as many times as needed (in the CSV file, 5000 sets have been generated, which details will be provided in Section 5). As common practice, we normalize $[\bar{K}]$ to obtain the normalized input matrix $[K]$, before proceeding.

4.1 Convolution and the generation of feature map, $[Z]$

Consider a typical normalized input matrix $[K]$ (from now on termed simply as input matrix) having a size of 6×6:

$$[K] = \begin{bmatrix} K_{11} & K_{12} & K_{13} & K_{14} & K_{15} & K_{16} \\ K_{21} & K_{22} & K_{23} & K_{24} & K_{25} & K_{26} \\ K_{31} & K_{32} & K_{33} & K_{34} & K_{35} & K_{36} \\ K_{41} & K_{42} & K_{43} & K_{44} & K_{45} & K_{46} \\ K_{51} & K_{52} & K_{53} & K_{54} & K_{55} & K_{56} \\ K_{61} & K_{62} & K_{63} & K_{64} & K_{65} & K_{66} \end{bmatrix} \quad (2)$$

Next, we are going to establish a kernel (or filter) of size 3 x 3, denoted as matrix $[W]$. Inside the kernel are the trainable weights, as given below:

$$[W] = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} \quad (3)$$

Then, we are going to convolve $[K]$ with $[W]$. We start by placing $[W]$ at the top-left corner of $[K]$, as shown in Figure 2.

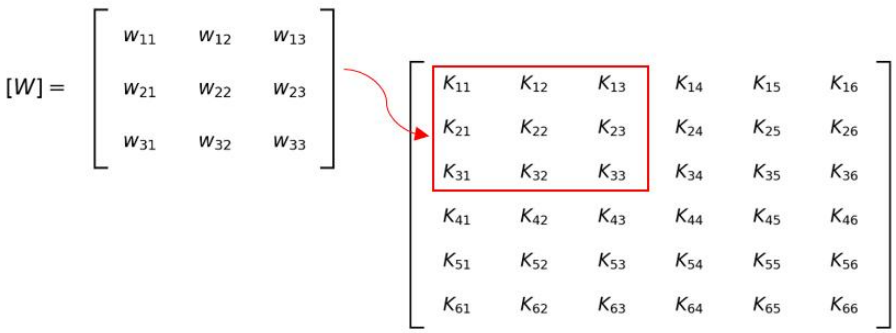


Figure 2: First convolution

This first convolution will produce the summing variable, or weighted sum Z_{11}, of the feature map $[Z]$ given as:

$$Z_{11} = K_{11}W_{11} + K_{12}W_{12} + K_{13}W_{13} + K_{21}W_{21} + K_{22}W_{22} + K_{23}W_{23} + K_{31}W_{31} + K_{32}W_{32} + K_{33}W_{33} + b \quad (4)$$

Observe the patterns in Eq. (4):

1) Convolution involves element-by-element multiplication between the kernel and the corresponding entries of the input matrix, followed by the summation of all the products with a bias, b, added to it.
2) It is in a similar form to the summing variable, Z, previously described in Module 1 for FFNN

We are going to repeat this process until the whole input matrix has been convolved by the kernel. Next, we slide the kernel one step to the right, corresponding to a stride of 1, as shown in Figure 3 below:

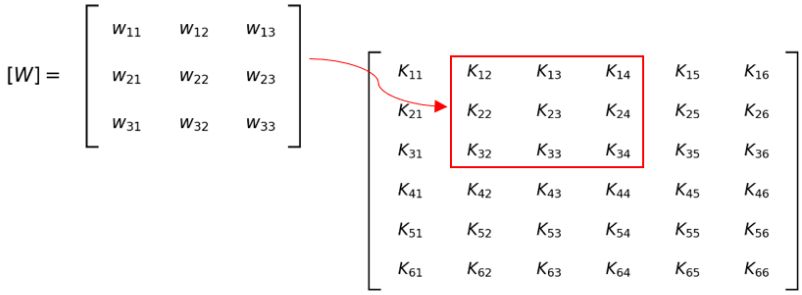


Figure 3: Second convolution

This will produce Z_{12}, given as:

$$Z_{12} = K_{12}W_{11} + K_{13}W_{12} + K_{14}W_{13} + K_{22}W_{21} + K_{23}W_{22} + K_{24}W_{23} + K_{32}W_{31} + K_{33}W_{32} + +K_{34}W_{33} + b \quad (5)$$

Observe that in Eq. (4), the same weights (i.e., W_{11}, W_{12}...,W_{33}) used in the previous convolution are carried over. Hence, these are termed “shared weights”. Weight sharing is a very important characteristic of CNN.

As mentioned, we are going to repeat this convolutional process until the kernel has covered the entire input matrix. This is demonstrated graphically in Figure 4 below:

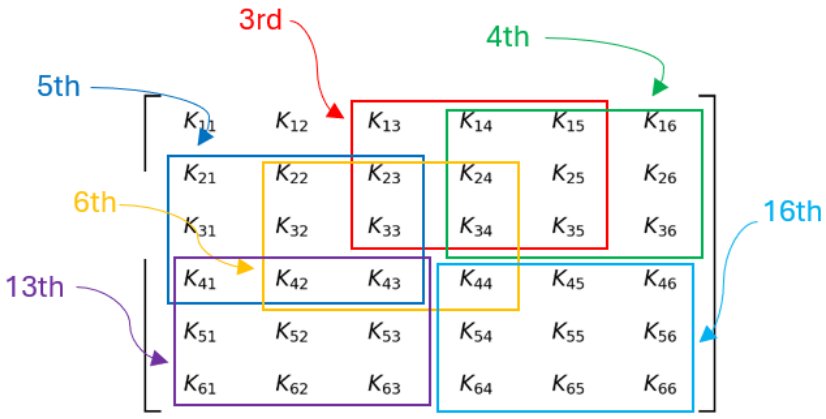


Figure 4: The remaining convolution process

The convolution process will produce the so-called feature map, $[Z]$, given as:

$$[Z] = \begin{bmatrix} z_{11} & z_{12} & z_{13} & z_{14} \\ z_{21} & z_{22} & z_{23} & z_{24} \\ z_{31} & z_{32} & z_{33} & z_{34} \\ z_{41} & z_{42} & z_{43} & z_{44} \end{bmatrix} \tag{6}$$

Before we proceed, let us discuss the significance of this action (convolution). If we scrutinize Eqs. (4) and (5), we can see that the equations have the same general form as that described for the summing variable, *Z* of FFNN in Module 1, except that, one, they are obtained from a local region of a matrix (instead of from a column of inputs), and two, the equations share the same weights (resulting from the same kernel convolving over the entire input matrix). But why do we want to do it this way? The answers are:

1. By convolving the input matrix using the same kernel weights, we interconnect neighbouring inputs within the kernel's coverage, allowing their local relationship patterns to be captured. If the input matrix contains positional information, this provides a mechanism for exploiting such spatial information. For example, in an image input matrix, say of a face, there is spatial information that the left ear is closer to the left eye than to the right eye, and this information is represented spatially by the locations of the corresponding entries in the input matrix.

   For another example, an input matrix obtained from an engineering matrix equation (i.e., stiffness matrix), for a given ordering of the global degrees of freedom, may contain information associated with the connectivity of elements resulting from nodal assembly. Therefore, the arrangement of the entries in the matrix may also carry structural information that can potentially be captured through local convolution (Herrmann and Kollmannsberger, 2024).

2. By sharing the weights, we reduce the number of learnable parameters, which can reduce the computational cost of training.

   To demonstrate how weight sharing can reduce the number of learnable parameters, let us consider our 6 x 6 input matrix. Our 3 x3 kernel contains 9 weights and one bias. Since the same weights and bias are shared throughout the convolution process, we only need 3 x 3 + 1 = 10 trainable parameters to generate the feature map, $[Z]$.

   Now, if we stick with the FFNN approach as discussed in Module 1, we first need to flatten our 6 x 6 input matrix into 36 x 1 input vector. If we want the first hidden layer to have the same number of neurons as the number of entries in our feature map, we would need 16 neurons. Since every input is connected to every neuron in an FFNN, we would then require (36 x 16) + 16 = 592 weight and biases, there would be a 98.3% reduction in the number of parameters if we opt for CNN!

Before we proceed to nonlinearization, note that, in our discussion above, we do not consider any padding (i.e., padding = 0).

4.2 Nonlinearization and the generation of the activated feature map, $[A]$

Next, we are going to nonlinearize our feature map, $[Z]$ by passing it through an activation function, say tanh, for the obvious reason. This will produce the so-called activated feature map, $[A]$, given as follows:

$$[A] = \begin{bmatrix} \tanh(z_{11}) & \tanh(z_{12}) & \tanh(z_{13}) & \tanh(z_{14}) \\ \tanh(z_{21}) & \tanh(z_{22}) & \tanh(z_{23}) & \tanh(z_{24}) \\ \tanh(z_{31}) & \tanh(z_{32}) & \tanh(z_{33}) & \tanh(z_{34}) \\ \tanh(z_{41}) & \tanh(z_{42}) & \tanh(z_{43}) & \tanh(z_{44}) \end{bmatrix}$$

$$= \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix} \quad (7)$$

4.3 Pooling and the generation of the pooled activated feature map, [*P*]

Pooling is an input-reduction technique in CNN. In terms of its general intention, we can relate it to the feature reduction/selection techniques that we commonly use in ML, such as PCA or feature selection based on a correlation heat map. Of course, they are mathematically different, but somehow the intention is similar; we want to reduce the amount of information to be carried forward while retaining the important features.

There are two common approaches for pooling, Average Pooling and Max Pooling. Average Pooling takes the average value within the pooling window, whilst Max Pooling takes the maximum value. In this module, we will opt for the latter.

There is another important observation to be made for pooling. For max pooling, since we choose only the maximum value, this also means that we ignore the rest of the values within the pooling window. This allowance of selecting some information while neglecting others can also be found in other ML approaches. For example, our network does not necessarily have to be fully dense or fully connected. We can even reduce the size of our network by pruning, that is, by permanently eliminating some of the less important weights, connections, or neurons (pruning is a model compression technique). Also, as one of the strategies to treat overfitting, we can drop some neurons during training through an approach called dropout.

Of course, these approaches work differently, but the point to highlight is that not all available information or connections necessarily need to be carried forward or utilized.

Now, let's come back and proceed with the pooling process. Let's say we consider a 2 x 2 pooling window. Like what we did with the kernel, we will place the pooling window, this time over the activated feature map, [*A*]. However, as a normal practice, when we slide the pooling window, we

will not overlap the previous region. In other words, we take the stride as equal to the size of the pooling window.

We start by placing the pooling window at the top-left corner of [*A*], as shown in Figure 5. As we can see, the pooling window is placed over the portion containing A_{11}, A_{12}, A_{21}, and A_{22}. For this window, we extract the largest value among all the entries and set it as P_{11}.

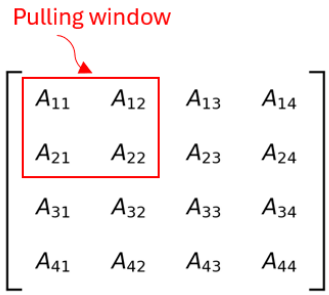


Figure 5: First max pooling

For the first max pooling with thus obtain:

$$P_{11} = \max(A_{11}, A_{12}, A_{21}, A_{22})$$

or

$$P_{11} = \max(\tanh(Z_{11}), \tanh(Z_{12}), \tanh(Z_{21}), \tanh(Z_{22})) \quad (8)$$

Figure 6 shows the second max pooling, where we slide the pooling window with a stride of 2 (the same size as the pooling window).

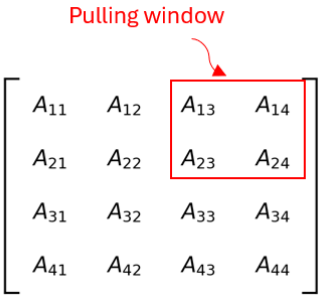


Figure 6: Second max pooling

For the second max pooling, we then obtain:

$$P_{12} = \max(A_{13}, A_{14}, A_{23}, A_{24})$$

or

$$P_{12} = \max(\tanh(Z_{13}), \tanh(Z_{14}), \tanh(Z_{23}), \tanh(Z_{24})) \quad (9)$$

We run two more pooling operations over the remaining portions of the activated feature map, as shown in Figure 7. By carrying out these remaining pooling operations, we will obtain the pooled activated feature map, $[P]$, as follows:

$$[P] = \begin{bmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{bmatrix} \tag{10}$$

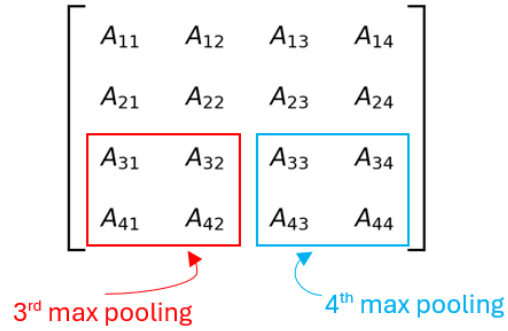


Figure 7: The remaining pooling operations

4.4 Flattening of $[P]$

Now that we have completed the pooling, we have two choices. Either we want to repeat the process by creating another convolutional stack (where we will start by convolving $[P]$ again), or we end the convolutional processing here and flatten $[P]$ so that it can become the input to the FFNN.

Since this module is introductory, we opt for the second choice. We will be content with one convolutional stack and will flatten $[P]$ so that it can become the input to the FFNN. The flattening of $[P]$ is given below. Figure 8 illustrates how the flattened $[P]$ is connected to the FFNN

$$\begin{bmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{bmatrix} \rightarrow \begin{Bmatrix} P_{11} \\ P_{12} \\ P_{21} \\ P_{22} \end{Bmatrix} \tag{11}$$

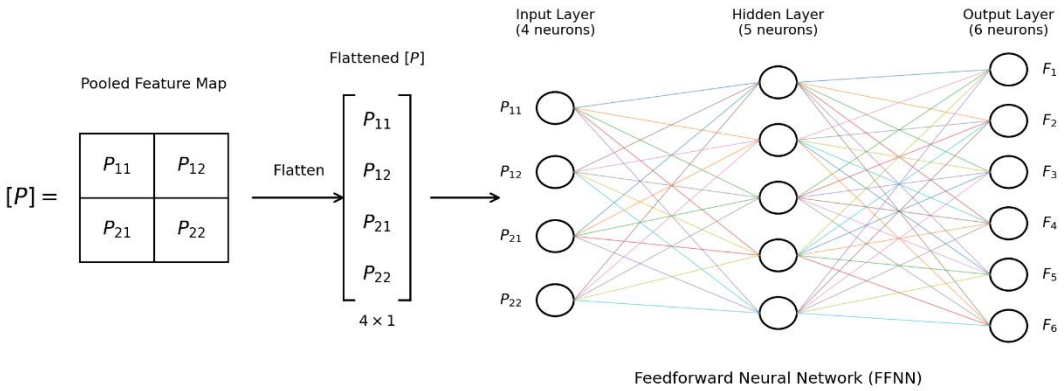


Figure 8: From convolutional layer to FFNN

5. CNN with PyTorch: Implementation of a Surrogate CNN+FFNN to Approximate a Hypothetical Finite Element Matrix System

Eq. (1) is typical finite element (FE) matrix system where the $[\bar{K}]$ is the global stiffness matrix, $\{\bar{U}\}$ is the global degree-of-freedom (dof) vector and $\{\bar{F}\}$ is the global load vector. In a typical FE process, after the imposition of boundary conditions, unknown dof vector $\{\bar{U}\}$ is solved for the known $[\bar{K}]$ and $\{\bar{F}\}$. This provides us with pairs of labelled data as follows:

$$\begin{array}{lll} \text{Inputs (features)} & & \text{Outputs (targets)} \\ ([\bar{K}],\{\bar{F}\}) & \longrightarrow & \{\bar{U}\} \end{array} \qquad (12)$$

where $[\bar{K}]$ and $\{\bar{F}\}$ serve as the inputs and $\{\bar{U}\}$ serves as the targets. Using Eq. (1) we generate 5000 such pairs of labelled data. The data are generated for the simply supported beam shown below, modelled using three Euler-Bernoulli beam elements. To introduce variations among the 5000 beam problems, the second moments of area of the three beam elements, I_1, I_2 and I_3, as well as the uniformly distributed loads (UDL), q_1, q_2 and q_3, are randomly varied, while the other beam properties are kept constant.

To note, what we are developing here can be termed an FEM surrogate model. A surrogate refers to a network or model that approximates a higher-fidelity model or an underlying mathematical relationship. In this case, our CNN+FFNN architecture approximates the FEM matrix system, specifically the relationship between $[\bar{K}]$, $\{\bar{F}\}$, and $\{\bar{U}\}$, and thus can be regarded as a surrogate model.

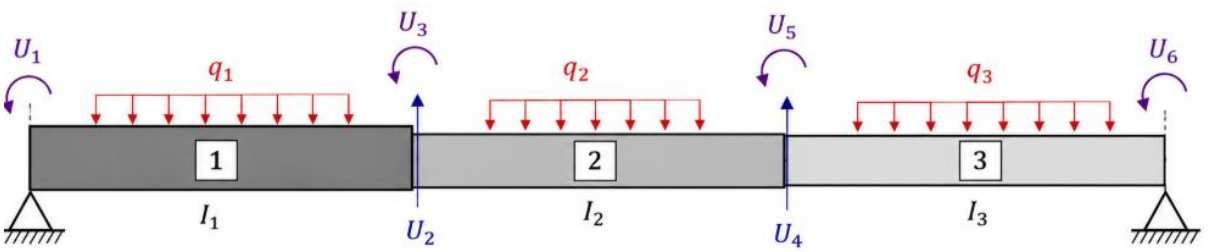


Figure 9: The beam arrangement for the generation of labelled data

Before we proceed, download the CNN_FEM_beam_K_F_to_U_5000.csv from Github (click link below):

https://github.com/msnm-official/ai_modules/blob/main/CNN_FEM_beam_K_F_to_U_5000.csv

5.1 CNN+FFNN Surrogate Architecture

Based on Eq. (12), we can see that we need to build a network having inputs from the normalized $[K]$, and $\{F\}$ with $\{U\}$ as the output. The normalized $[K]$ will first go through the convolutional process before being flattened and uses as input to the FFNN. The normalized $\{F\}$, on the other hand, will be inserted directly into the FFNN as another input. Such an architecture is shown in Figure 10 below.

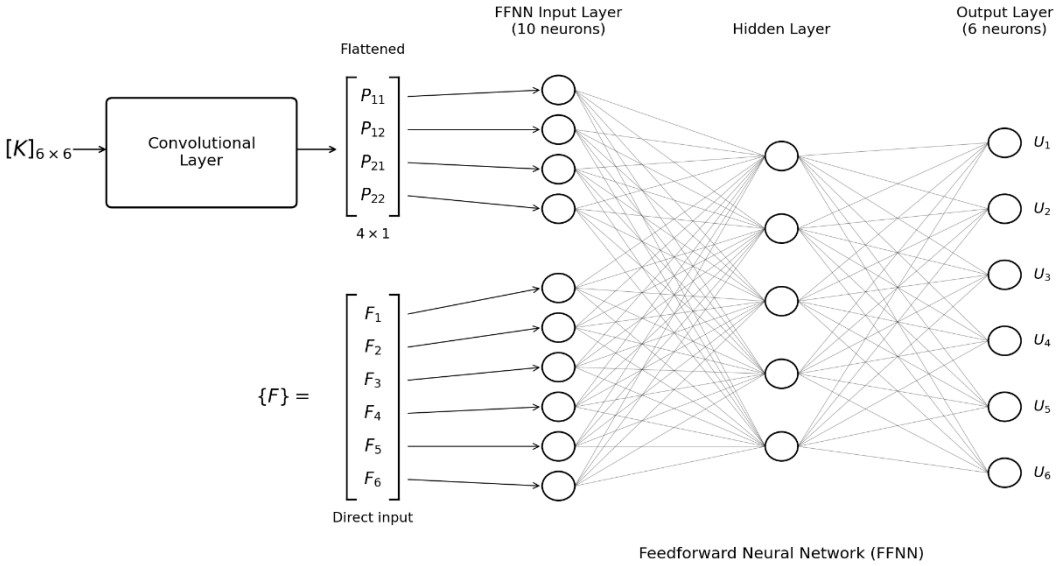


Figure 10: The CNN+FFNN surrogate architecture for the hypothetical FEM matrix system

Next, we will discuss the steps to build such a surrogate model.

5.2 Step-by-step PyTorch Implementation

Next, we will discuss the steps to build the surrogate model, as summarized in Figure 11.

Step 1: Import libraries

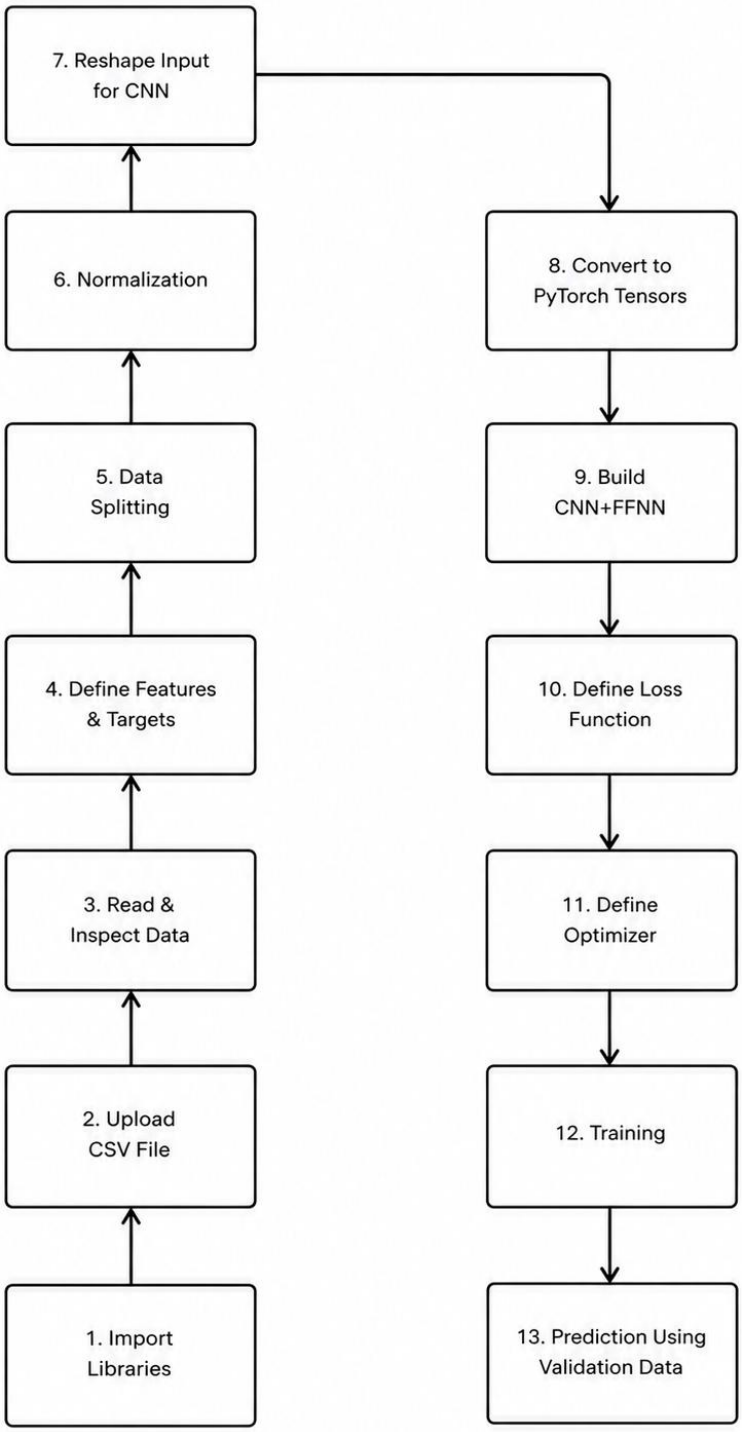
The main libraries that we will use are:

- NumPy (for numerical and array operations)
- Pandas (for reading and handling our dataset)
- PyTorch (for building and training the NN)
- Matplotlib (for plotting and visualization)
- Scikit-learn (for data preprocessing and other ML procedures)

Copyable code:

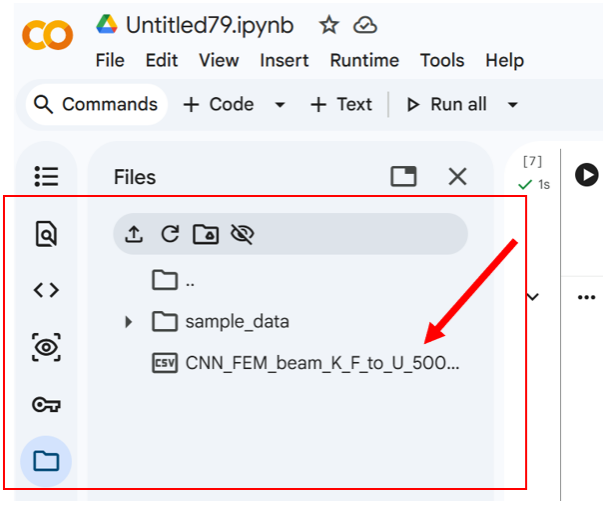
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Figure 11: CNN+FFNN Surrogate Model Workflow with PyTorch



Step 2: Upload CSV file

Before the data can be read and processed by our Python code, we first must upload the CSV file into our Google Colab. To do this, click the Files icon (folder icon) on the left panel of Google Colab. Then, click the Upload icon and select the file CNN_FEM_beam_K_F_to_U_5000.csv from your computer. Once uploaded, the file should appear in the Files panel. The CSV file is now available in the current Colab session and can be read by our Python code in the next step.



Step 3: Read and inspect data

To read a CSV file, we use the Pandas function pd.read_csv(). To inspect the data, we request to see the heading of the table in the DataFrame and the first 5 rows using function print(data.head()). We can also request the size of the data using function print("\nDataset shape:", data.shape).

Copyable code:

```
data = pd.read_csv('CNN_FEM_beam_K_F_to_U_5000.csv')

print(data.head())

print("\nDataset shape:", data.shape)
```

If we run the code, we will see the output below. As we can see, each row (instance) consists of 36 entries of the raw input matrix, $[\bar{K}]$, followed by the 6 entries of the raw load vector, $\{\bar{F}\}$ which followed by 6 entries of the raw output DOF vector, $\{\bar{U}\}$. The entries of $[\bar{K}]$ are arranged in the DataFrame as $K_{11}, K_{12,} \ldots K_{66}$, the entries of $\{\bar{F}\}$ are arranged as $F_1, F_{2,} \ldots F_6$ whilst the entries of $\{\bar{U}\}$ are arranged as $U_1, U_{2,} \ldots U_6$. Therefore, each row (instance) contains 42 features and 6 targets. Note that the entries are denoted without the bar-hat notation because Python does not support such notation. Thus, we must recognize from the context that these are the raw data before normalization.

Finally, the DataFrame has a shape of (5000, 48) as expected, since 5000 instances (samples) have been randomly generated, each consisting of 42 features and 6 targets.

```
data = pd.read_csv('CNN_FEM_beam_K_F_to_U_5000.csv')
print(data.head())
print("\nDataset shape:", data.shape)
```

```
             K11           K12           K13  K14  K15  K16           K21  \
0  3.186242e+07 -2.389681e+07  1.593121e+07  0.0  0.0  0.0 -2.389681e+07
1  3.140103e+07 -2.355077e+07  1.570051e+07  0.0  0.0  0.0 -2.355077e+07
2  2.717914e+07 -2.038436e+07  1.358957e+07  0.0  0.0  0.0 -2.038436e+07
3  3.379472e+07 -2.534604e+07  1.689736e+07  0.0  0.0  0.0 -2.534604e+07
4  3.202181e+07 -2.401635e+07  1.601090e+07  0.0  0.0  0.0 -2.401635e+07

             K22           K23           K24  ...           F3            F4  \
0  3.874653e+07 -9.047095e+06 -1.484972e+07  ...  9047.860218 -58140.986479
1  4.777451e+07  6.729643e+05 -2.422374e+07  ...  1193.818705 -68390.335589
2  4.559892e+07  4.830205e+06 -2.521456e+07  ... -4910.190978 -37828.091940
3  4.540098e+07 -5.291103e+06 -2.005494e+07  ... -9242.580844 -93871.861557
4  3.227160e+07 -1.576111e+07 -8.255245e+06  ... -1657.285894 -47680.230039

              F5            F6        U1        U2        U3        U4  \
0 -13221.675056  16301.001941 -0.018561 -0.031433 -0.010788 -0.031705
1  -8339.504469  15568.141499 -0.021321 -0.036975 -0.013357 -0.048805
2   7361.512964   2623.925508 -0.012943 -0.020947 -0.005908 -0.021601
3   1163.653546  15063.483486 -0.023977 -0.040996 -0.013952 -0.042416
4  -7931.391918  11912.400965 -0.017651 -0.032638 -0.013800 -0.035752

         U5        U6
0  0.010796  0.018848
1  0.001823  0.037500
2  0.005157  0.013754
3  0.012934  0.025827
4  0.011847  0.021463

[5 rows x 48 columns]

Dataset shape: (5000, 48)
```

Step 4: Define features and targets

Compared to Module 2, instead of writing the column names directly inside data[' '], we first define them using the variables feature_columns and target_columns, and then insert these variables as the arguments. This is simply to shorten the expression of each statement.

Copyable code:

```
K_columns = [
  'K11', 'K12', 'K13', 'K14', 'K15', 'K16',
  'K21', 'K22', 'K23', 'K24', 'K25', 'K26',
  'K31', 'K32', 'K33', 'K34', 'K35', 'K36',
  'K41', 'K42', 'K43', 'K44', 'K45', 'K46',
  'K51', 'K52', 'K53', 'K54', 'K55', 'K56',
  'K61', 'K62', 'K63', 'K64', 'K65', 'K66'
]
F_columns = [
```

```
   'F1', 'F2', 'F3', 'F4', 'F5', 'F6'
]
U_columns = [
   'U1', 'U2', 'U3', 'U4', 'U5', 'U6'
]
K_data = data[K_columns].values
F_data = data[F_columns].values
U_data = data[U_columns].values
print("K shape:", K_data.shape)
print("F shape:", F_data.shape)
print("U shape:", U_data.shape)
```

If we run the code, we will see this output:

```
K_columns = [
    'K11', 'K12', 'K13', 'K14', 'K15', 'K16',
    'K21', 'K22', 'K23', 'K24', 'K25', 'K26',
    'K31', 'K32', 'K33', 'K34', 'K35', 'K36',
    'K41', 'K42', 'K43', 'K44', 'K45', 'K46',
    'K51', 'K52', 'K53', 'K54', 'K55', 'K56',
    'K61', 'K62', 'K63', 'K64', 'K65', 'K66'
]
F_columns = [
    'F1', 'F2', 'F3', 'F4', 'F5', 'F6'
]
U_columns = [
    'U1', 'U2', 'U3', 'U4', 'U5', 'U6'
]
K_data = data[K_columns].values
F_data = data[F_columns].values
U_data = data[U_columns].values
print("K shape:", K_data.shape)
print("F shape:", F_data.shape)
print("U shape:", U_data.shape)
```

```
K shape: (5000, 36)
F shape: (5000, 6)
U shape: (5000, 6)
```

From the printed shapes, we can see that the NumPy arrays have been created and their shapes are correct.

Step 5: Data splitting

We split all the data into training sets (70%) and validation sets (30%). As mentioned in Module 2, we use such a splitting because we are not concerning ourselves with hyperparameter tuning for now.

Copyable code:

```
split = int(0.7 * len(K_data))
K_train = K_data[:split]
F_train = F_data[:split]
U_train = U_data[:split]
K_val = K_data[split:]
F_val = F_data[split:]
U_val = U_data[split:]
print("K_train shape:", K_train.shape)
print("F_train shape:", F_train.shape)
print("U_train shape:", U_train.shape)
print("K_val shape  :", K_val.shape)
print("F_val shape  :", F_val.shape)
print("U_val shape  :", U_val.shape)
```

If we run the code, we will see this output:

```
split = int(0.7 * len(K_data))
K_train = K_data[:split]
F_train = F_data[:split]
U_train = U_data[:split]
K_val = K_data[split:]
F_val = F_data[split:]
U_val = U_data[split:]
print("K_train shape:", K_train.shape)
print("F_train shape:", F_train.shape)
print("U_train shape:", U_train.shape)
print("K_val shape  :", K_val.shape)
print("F_val shape  :", F_val.shape)
print("U_val shape  :", U_val.shape)

K_train shape: (3500, 36)
F_train shape: (3500, 6)
U_train shape: (3500, 6)
K_val shape  : (1500, 36)
F_val shape  : (1500, 6)
U_val shape  : (1500, 6)
```

From the printed shapes, we can see that the splitting has been successful, where the sample sizes are appropriate.

Step 6: Normalization

Essentially, there is no difference in the procedure and the code as was given in Module 2, where the process is automated using the MinMaxScaler class. The maximum and minimum values are still taken from the training sets, hence the use of fit_transform() for K_train, F_train and U_train, and transform() for K_val, F_val and U_val.

Copyable code:

```
scaler_K = MinMaxScaler()

scaler_F = MinMaxScaler()

scaler_U = MinMaxScaler()

K_train_norm = scaler_K.fit_transform(K_train)

K_val_norm   = scaler_K.transform(K_val)

F_train_norm = scaler_F.fit_transform(F_train)

F_val_norm   = scaler_F.transform(F_val)

U_train_norm = scaler_U.fit_transform(U_train)

U_val_norm   = scaler_U.transform(U_val)

print("Before normalization:")

print("K_train[0] =", K_train[0])

print("\nAfter normalization:")

print("K_train_norm[0] =", K_train_norm[0])
```

If we run the code, we will see this output:

```
scaler_K = MinMaxScaler()
scaler_F = MinMaxScaler()
scaler_U = MinMaxScaler()
K_train_norm = scaler_K.fit_transform(K_train)
K_val_norm   = scaler_K.transform(K_val)
F_train_norm = scaler_F.fit_transform(F_train)
F_val_norm   = scaler_F.transform(F_val)
U_train_norm = scaler_U.fit_transform(U_train)
U_val_norm   = scaler_U.transform(U_val)
print("Before normalization:")
print("K_train[0] =", K_train[0])
print("\nAfter normalization:")
print("K_train_norm[0] =", K_train_norm[0])
```

```
Before normalization:
K_train[0] = [ 31862417.74801468 -23896813.31101101  15931208.87400734
         0.                0.                0.
 -23896813.31101101  38746531.18431643  -9047095.4377056
 -14849717.87330541  14849717.87330541         0.
  15931208.87400734  -9047095.4377056   51662041.57908857
 -14849717.87330541   9899811.91553694         0.
         0.         -14849717.87330541 -14849717.87330541
  41031861.71091274  11332425.96430191  26182143.83760733
         0.          14849717.87330541   9899811.91553694
  11332425.96430191  54709148.94788365  17454762.55840489
         0.                0.                0.
  26182143.83760733  17454762.55840489  34909525.11680977]

After normalization:
K_train_norm[0] = [0.77390235 0.22609765 0.77390235 0.         0.         0.
 0.22609765 0.61027405 0.33058052 0.56108376 0.43891624 0.
 0.77390235 0.33058052 0.61027405 0.56108376 0.43891624 0.
 0.         0.56108376 0.56108376 0.64865944 0.71110368 0.85856243
 0.         0.43891624 0.43891624 0.71110368 0.64865944 0.85856243
 0.         0.         0.         0.85856243 0.85856243 0.85856243]
```

From the printed values, we can see that the normalization has been successful, where none of the values of K_train_norm are below zero or exceed one.

Step 7: Reshape input for CNN

This is a new step compared with Module 2. In PyTorch, to build CNN, we will use Conv2d class. It expects input in the form of (number of instances, number of channels, height of matrix, width of matrix). For our training data, number of instances = 3500, number of channels = 1 (for RGB images, this will be 3), height = 6 and width = 6. Note that, only $[K]$ is reshaped because only $[K]$ passes through the convolutional layer. $\{F\}$ does not need reshaping because it will be inserted directly into the FFNN.

So, we need to only reshape K_train_norm and K_val_norm using reshape() function.

Copyable code:

```
K_train_norm = K_train_norm.reshape(-1, 1, 6, 6)

K_val_norm   = K_val_norm.reshape(-1, 1, 6, 6)

print("K_train_norm shape:", K_train_norm.shape)

print("K_val_norm shape  :", K_val_norm.shape)
```

If we run the code, we will see this output:

```
K_train_norm = K_train_norm.reshape(-1, 1, 6, 6)
K_val_norm   = K_val_norm.reshape(-1, 1, 6, 6)
print("K_train_norm shape:", K_train_norm.shape)
print("K_val_norm shape  :", K_val_norm.shape)
```

```
K_train_norm shape: (3500, 1, 6, 6)
K_val_norm shape  : (1500, 1, 6, 6)
```

From the printed shapes, we can see that the K_train_norm and K_val_norm have been reshaped into the intended CNN input form.

Step 8: Convert to PyTorch tensors

However, since we have decided to use PyTorch to build our NN architecture, we must convert our data into a format that can be read and handled by PyTorch. For this, we use the PyTorch function torch.tensor(). The resulting data are called PyTorch tensors. A tensor can be considered as a data structure used by PyTorch to store and perform operations on numerical data.

Copyable code:

```
K_train_tensor = torch.tensor(
  K_train_norm,
  dtype=torch.float32
)
K_val_tensor = torch.tensor(
  K_val_norm,
  dtype=torch.float32
)
F_train_tensor = torch.tensor(
  F_train_norm,
  dtype=torch.float32
)
F_val_tensor = torch.tensor(
  F_val_norm,
  dtype=torch.float32
)
U_train_tensor = torch.tensor(
  U_train_norm,
```

```
    dtype=torch.float32
)
U_val_tensor = torch.tensor(
    U_val_norm,
    dtype=torch.float32
)


print("K_train_tensor:", K_train_tensor.size())
print("K_val_tensor  :", K_val_tensor.size())
print("F_train_tensor:", F_train_tensor.size())
print("F_val_tensor  :", F_val_tensor.size())
print("U_train_tensor:", U_train_tensor.size())
print("U_val_tensor  :", U_val_tensor.size())
```

If we run the code, we will see this output:

```
K_train_tensor = torch.tensor(
    K_train_norm,
    dtype=torch.float32
)
K_val_tensor = torch.tensor(
    K_val_norm,
    dtype=torch.float32
)
F_train_tensor = torch.tensor(
    F_train_norm,
    dtype=torch.float32
)
F_val_tensor = torch.tensor(
    F_val_norm,
    dtype=torch.float32
)
U_train_tensor = torch.tensor(
    U_train_norm,
    dtype=torch.float32
)
U_val_tensor = torch.tensor(
    U_val_norm,
    dtype=torch.float32
)

print("K_train_tensor:", K_train_tensor.size())
print("K_val_tensor  :", K_val_tensor.size())
print("F_train_tensor:", F_train_tensor.size())
print("F_val_tensor  :", F_val_tensor.size())
print("U_train_tensor:", U_train_tensor.size())
print("U_val_tensor  :", U_val_tensor.size())
```

```
K_train_tensor: torch.Size([3500, 1, 6, 6])
K_val_tensor  : torch.Size([1500, 1, 6, 6])
F_train_tensor: torch.Size([3500, 6])
F_val_tensor  : torch.Size([1500, 6])
U_train_tensor: torch.Size([3500, 6])
U_val_tensor  : torch.Size([1500, 6])
```

From the printed sizes, we can see that the conversion has been successful, where the sample sizes are as expected.

Step 9: Build CNN+FNN architecture

This is the stage where we build our CNN+FNN surrogate model, consisting of two parts, the convolutional layer and the FFNN. To build the CNN, we use class CNN(nn.Module). Within this class, we use nn.Conv2d() which is a class that creates the convolutional layer to carry out the matrix convolution operation described earlier.

For a single channel like ours, in_channels=1 whilst for, say an RGB, it would be 3. In our discussion, we opt for only one feature map from a single kernel, thus out_channels=1. However, if we want, say 3 feature maps, we need three kernels, each having a different set of weights, thus out_channels would be 3. We have discussed that we will opt for 3 x 3 kernel, a single stride, and a zero padding hence the value written in the remaining arguments of nn.Conv2d().

To carry out max pooling, we use the class nn.MaxPool2d() and specify the kernel (pooling window) size and the stride.

We then set the FFNN by specifying the connection from the flattened pooled feature map plus the six inputs from the load vector to the hidden layer (32 neurons), and then from the hidden layer to the output layer consisting of six outputs representing the dof. The setup is:

self.fc1 = nn.Linear(10, 32)

self.fc2 = nn.Linear(32, 6)

The FFNN input is setup by concatenating the flattened pooled feature map and the values of the load vector using:

x = torch.cat((x, F), dim=1)

We employ a single hidden layer for now, but to have more hidden layers, we can proceed in the same way as we did in Module 2.

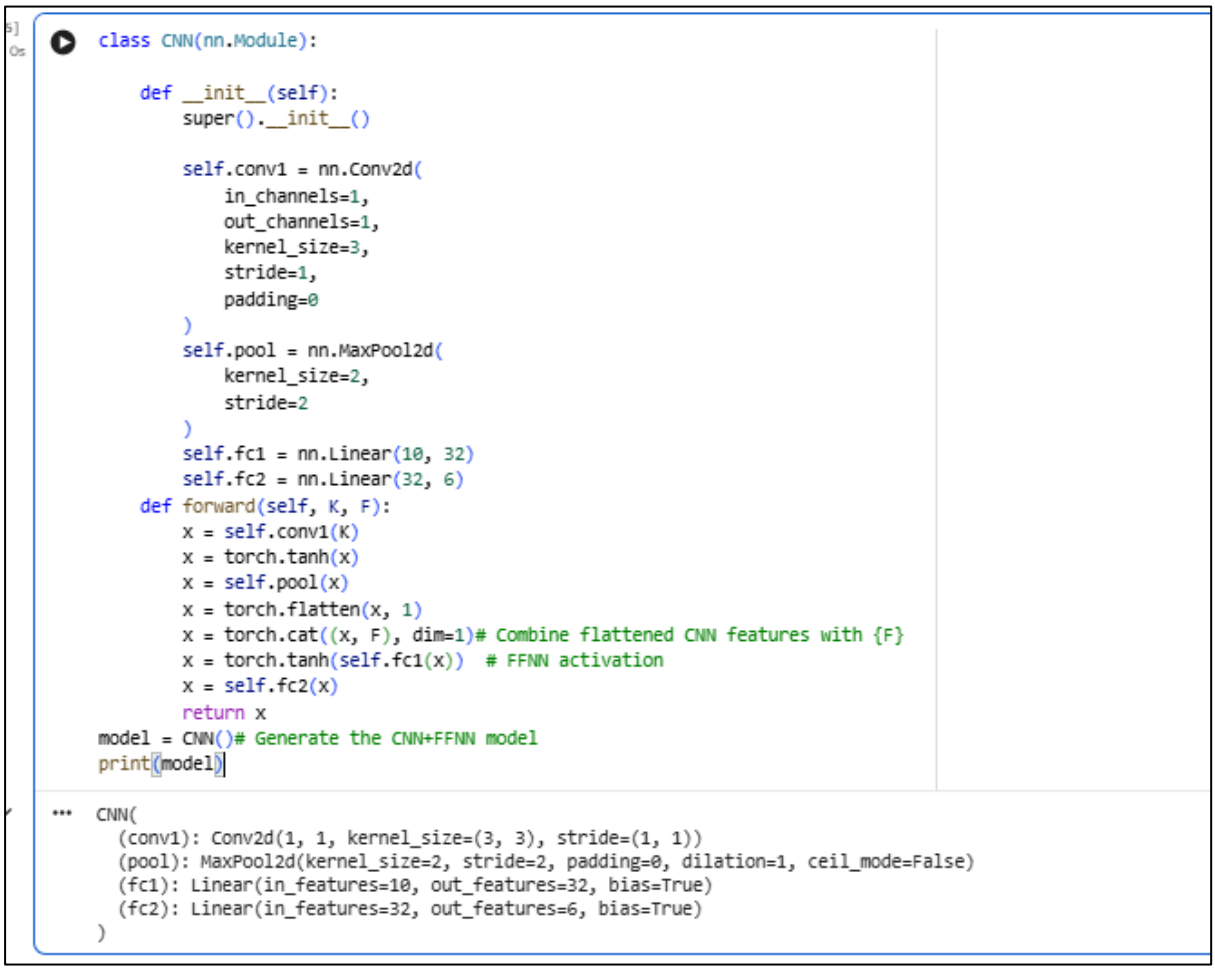
Finally, we defined the forward passing of the input, from convolution (x = self.conv1(x)), to nonlinearization (x = torch.tanh(x)), to pooling (x = self.pool(x)), to flattening (x = torch.flatten(x, 1)), to nonlinearization in the FFNN hidden layer (x = torch.tanh(self.fc1(x))) and finally to the output layer (x = self.fc2(x)).

Copyable code:

```
class CNN(nn.Module):


  def __init__(self):
    super().__init__()


    self.conv1 = nn.Conv2d(
      in_channels=1,
      out_channels=1,
      kernel_size=3,
      stride=1,
      padding=0
    )
    self.pool = nn.MaxPool2d(
      kernel_size=2,
      stride=2
    )
    self.fc1 = nn.Linear(10, 32)
```

```
    self.fc2 = nn.Linear(32, 6)

  def forward(self, K, F):

    x = self.conv1(K)

    x = torch.tanh(x)

    x = self.pool(x)

    x = torch.flatten(x, 1)

    x = torch.cat((x, F), dim=1)# Combine flattened CNN features with {F}

    x = torch.tanh(self.fc1(x))  # FFNN activation

    x = self.fc2(x)

    return x

model = CNN()# Generate the CNN+FFNN model

print(model)
```

If we run the code, we will see this input:



```
class CNN(nn.Module):

    def __init__(self):
        super().__init__()

        self.conv1 = nn.Conv2d(
            in_channels=1,
            out_channels=1,
            kernel_size=3,
            stride=1,
            padding=0
        )
        self.pool = nn.MaxPool2d(
            kernel_size=2,
            stride=2
        )
        self.fc1 = nn.Linear(10, 32)
        self.fc2 = nn.Linear(32, 6)
    def forward(self, K, F):
        x = self.conv1(K)
        x = torch.tanh(x)
        x = self.pool(x)
        x = torch.flatten(x, 1)
        x = torch.cat((x, F), dim=1)# Combine flattened CNN features with {F}
        x = torch.tanh(self.fc1(x))  # FFNN activation
        x = self.fc2(x)
        return x
model = CNN()# Generate the CNN+FFNN model
print(model)
```

```
CNN(
  (conv1): Conv2d(1, 1, kernel_size=(3, 3), stride=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=10, out_features=32, bias=True)
  (fc2): Linear(in_features=32, out_features=6, bias=True)
)
```

From the printed output, we can see that our CNN architecture has been successfully generated, consistent with the architecture illustrated earlier in Figure 10.

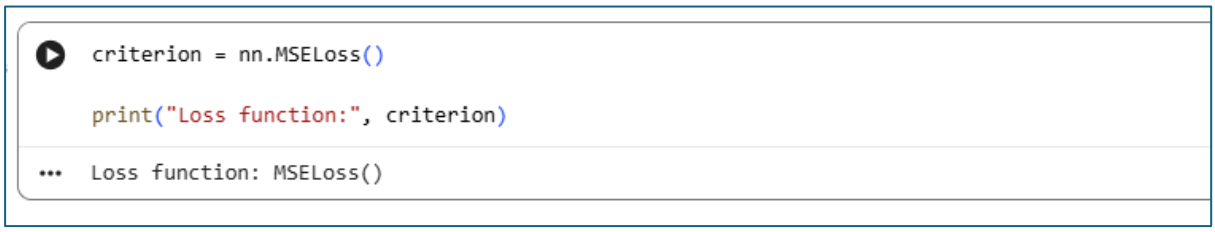
Step 10: Define loss function

Since our problem is still a regression problem, there is no change to the choice of loss function compared with Module 2. We continue to use the Mean Squared Error (MSE) by employing criterion = nn.MSELoss().

Copyable code:

```
criterion = nn.MSELoss()

print("Loss function:", criterion)
```

If we run the code, we will see this output:



From the printed output, we can see that MSE has been set up as the loss function.

Step 11: Define optimizer

Similar to the loss function, essentially there is no change to the optimizer compared with Module 2. But we will employ Adam with a learning rate of 0.001. However just remember that, model.parameters() now includes not only the weights and biases in the FFNN, but also the learnable weights and bias of the kernel in the convolutional layer. Therefore, all these parameters will be updated by the optimizer during training.

Copyable code:

```
learning_rate = 0.001

optimizer = optim.Adam(
   model.parameters(),
   lr=learning_rate
)
```

Run the code, but don't expect any output because we do not ask the code to print or display anything.

Step 12: Training

Essentially, there is no change to the code given in Module 2. To execute the training means we need to instruct the following process:

i. the forward pass
ii. the loss calculation
iii. the loss gradient calculation
iv. the parameter update

The model now receives two inputs, K_train_tensor and F_train_tensor, and predicts U (i.e. U_pred). The loss is then calculated from the error between U_pred and U_train_tensor. The rest of the lines remain as in Module 2.

Copyable code:

```
from IPython.display import clear_output

epochs = 5000

loss_history = []

for epoch in range(epochs):
  # Forward pass
  U_pred = model(
    K_train_tensor,
    F_train_tensor
  )


  # Loss calculation
  loss = criterion(
    U_pred,
    U_train_tensor
  )
  optimizer.zero_grad()   # Loss gradient calculation
  loss.backward()
  optimizer.step()# Parameter update
  loss_history.append(loss.item())# Record loss
  # Live plot
  if (epoch + 1) % 100 == 0:
```

```
    clear_output(wait=True)
    plt.figure(figsize=(8, 5))
    plt.plot(loss_history)
    plt.xlabel("Epoch")
    plt.ylabel("MSE Loss")
    plt.title("CNN+FFNN Surrogate Training Loss")
    plt.grid(True)
    plt.show()
    print(
      f"Epoch [{epoch+1}/{epochs}], "
      f"Loss: {loss.item():.8f}"
    )
```

By running the code, we would obtain the convergence curve as shown in Figure 12. As can be seen, the loss reduces as the number of epochs increases. This indicates that loss is being minimized and the errors are diminishing, hence showing convergence.

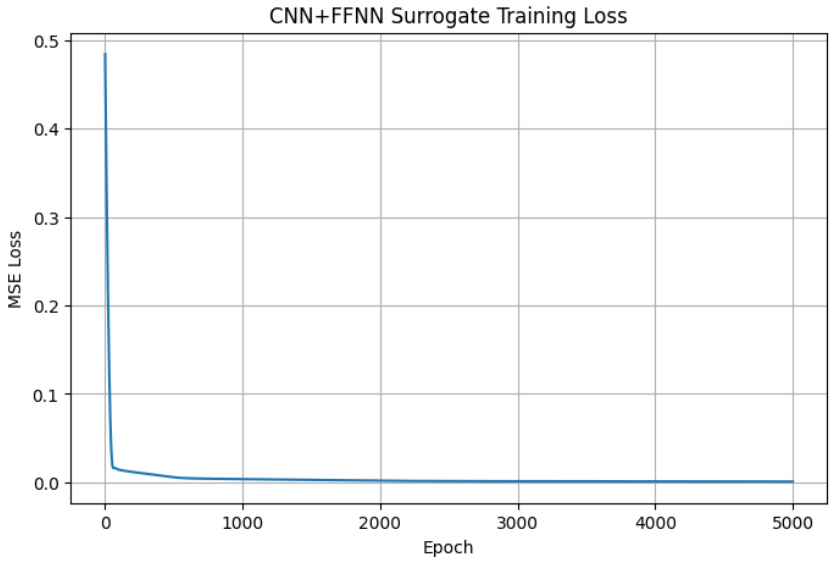


Figure 12: Convergence of the Loss

Step 13: Prediction using validation data

Now that we have completed the training, it's time to check whether our model can really make predictions. For this, we will use the remaining 30% of the dataset, which we have specified as K_val and F_val and later transformed into PyTorch tensors, denoted as K_val_tensor and F_val_tensor, respectively. They will be used to produce the predicted values, U_val_pred_norm, which are later denormalized or inverse-transformed into U_val_pred.

To make the prediction, we use model.eval() together with torch.no_grad() to tell PyTorch that we are not training anymore (thus, we don't need the gradients).

We don't need to inverse-transform U_val because it is already in its original magnitude.

So, the comparison will be U_val_pred vs U_val.

Copyable code (for prediction):

```
model.eval()
with torch.no_grad():
  U_val_pred_norm = model(
    K_val_tensor,
    F_val_tensor
  )
U_val_pred_norm = U_val_pred_norm.numpy()
U_val_pred = scaler_U.inverse_transform(
  U_val_pred_norm
)
```

Now, the code above is about calculating the predicted values of the outputs, we are yet to compare them against the actual values. To make such a comparison, we can either calculate the difference between U_val_pred vs U_val (in the form of performance metrics, MSE, RMSE, MAE and R^2) or create a scatter plot where the coordinates of each data point are (U_val, U_val_pred) and assess how far the points are from the 45-degree reference line.

A quick check on the model performance can be made by looking at the MAE and RMSE, where relatively small values indicate a better-performing model. For R^2, a value closer to 1 indicates a better agreement between the actual and predicted values. Also, for a good-performing model, the scattered data points should be very close or in proximity to the 45-degree reference line.

Copyable code (creating the predicted-actual pair for the 6 outputs):

```
actual_U1 = U_val[:, 0]
predicted_U1 = U_val_pred[:, 0]

actual_U2 = U_val[:, 1]
predicted_U2 = U_val_pred[:, 1]

actual_U3 = U_val[:, 2]
predicted_U3 = U_val_pred[:, 2]

actual_U4 = U_val[:, 3]
predicted_U4 = U_val_pred[:, 3]

actual_U5 = U_val[:, 4]
predicted_U5 = U_val_pred[:, 4]

actual_U6 = U_val[:, 5]
predicted_U6 = U_val_pred[:, 5]
```

Copyable code: Calculating performance (MSE, RMSE, MAE, R^2)

```
MSE_U1 = mean_squared_error(actual_U1, predicted_U1)
RMSE_U1 = np.sqrt(MSE_U1)
MAE_U1 = mean_absolute_error(actual_U1, predicted_U1)
R2_U1 = r2_score(actual_U1, predicted_U1)


# Performance metrics for U2
MSE_U2 = mean_squared_error(actual_U2, predicted_U2)
RMSE_U2 = np.sqrt(MSE_U2)
MAE_U2 = mean_absolute_error(actual_U2, predicted_U2)
R2_U2 = r2_score(actual_U2, predicted_U2)

# Performance metrics for U3
MSE_U3 = mean_squared_error(actual_U3, predicted_U3)
RMSE_U3 = np.sqrt(MSE_U3)
MAE_U3 = mean_absolute_error(actual_U3, predicted_U3)
R2_U3 = r2_score(actual_U3, predicted_U3)

# Performance metrics for U4
MSE_U4 = mean_squared_error(actual_U4, predicted_U4)
RMSE_U4 = np.sqrt(MSE_U4)
MAE_U4 = mean_absolute_error(actual_U4, predicted_U4)
R2_U4 = r2_score(actual_U4, predicted_U4)

# Performance metrics for U5
MSE_U5 = mean_squared_error(actual_U5, predicted_U5)
RMSE_U5 = np.sqrt(MSE_U5)
MAE_U5 = mean_absolute_error(actual_U5, predicted_U5)
R2_U5 = r2_score(actual_U5, predicted_U5)

# Performance metrics for U6
MSE_U6 = mean_squared_error(actual_U6, predicted_U6)
RMSE_U6 = np.sqrt(MSE_U6)
MAE_U6 = mean_absolute_error(actual_U6, predicted_U6)
R2_U6 = r2_score(actual_U6, predicted_U6)
```

Copyable code: Print MSE, RMSE, MAE, R^2

```
print("Performance for output U1")
print(f"MSE  = {MSE_U1:.8e}")
print(f"RMSE = {RMSE_U1:.8e}")
print(f"MAE  = {MAE_U1:.8e}")
print(f"R²   = {R2_U1:.6f}")

print("\nPerformance for output U2")
print(f"MSE  = {MSE_U2:.8e}")
print(f"RMSE = {RMSE_U2:.8e}")
print(f"MAE  = {MAE_U2:.8e}")
print(f"R²   = {R2_U2:.6f}")
```

```
print("\nPerformance for output U3")
print(f"MSE  = {MSE_U3:.8e}")
print(f"RMSE = {RMSE_U3:.8e}")
print(f"MAE  = {MAE_U3:.8e}")
print(f"R²   = {R2_U3:.6f}")

print("\nPerformance for output U4")
print(f"MSE  = {MSE_U4:.8e}")
print(f"RMSE = {RMSE_U4:.8e}")
print(f"MAE  = {MAE_U4:.8e}")
print(f"R²   = {R2_U4:.6f}")

print("\nPerformance for output U5")
print(f"MSE  = {MSE_U5:.8e}")
print(f"RMSE = {RMSE_U5:.8e}")
print(f"MAE  = {MAE_U5:.8e}")
print(f"R²   = {R2_U5:.6f}")

print("\nPerformance for output U6")
print(f"MSE  = {MSE_U6:.8e}")
print(f"RMSE = {RMSE_U6:.8e}")
print(f"MAE  = {MAE_U6:.8e}")
print(f"R²   = {R2_U6:.6f}")
```

If we run all the codes above, we will see the following output:

```
Performance for output U1
MSE  = 3.79686538e-05
RMSE = 6.16187096e-03
MAE  = 4.45589063e-03
R²   = 0.877156

Performance for output U2
MSE  = 4.88994450e-05
RMSE = 6.99281381e-03
MAE  = 5.10795826e-03
R²   = 0.943338

Performance for output U3
MSE  = 7.49450959e-06
RMSE = 2.73761020e-03
MAE  = 1.82739677e-03
R²   = 0.952464

Performance for output U4
MSE  = 3.20869585e-05
RMSE = 5.66453515e-03
MAE  = 4.08434846e-03
R²   = 0.962660

Performance for output U5
MSE  = 4.70423362e-06
RMSE = 2.16892453e-03
MAE  = 1.57524602e-03
R²   = 0.970079

Performance for output U6
MSE  = 1.88882402e-05
RMSE = 4.34606031e-03
MAE  = 3.01311312e-03
R²   = 0.938008
```

From the output, we can see the performance metrics for all six outputs. As mentioned, a quick check on the model performance can be made by looking at the MAE, RMSE and R^2, where relatively small values of the former two or values close to 1 for the latter, indicate a good-performing model.

Based on the values, especially the R^2 ranging from 0.877 to 0.970, we can say that our model is performing very well, even before we carry out any hyper parameter tuning.

Next, we can visually assess the performance of the model by plotting the scatter plots.

Copyable code: Scatter plots

```
for i in range(6):

  actual = U_val[:, i]
  predicted = U_val_pred[:, i]

  R2 = r2_score(
    actual,
    predicted
  )
  min_val = min(
    actual.min(),
    predicted.min()
```

```
  )
  max_val = max(
    actual.max(),
    predicted.max()
  )
  plt.figure(figsize=(6, 6))

  plt.scatter(
    actual,
    predicted,
    alpha=0.5
  )
  # 45-degree reference line
  plt.plot(
    [min_val, max_val],
    [min_val, max_val],
    '--',
    label='45° Reference Line'
  )
  plt.xlabel(f"Actual U{i+1}")
  plt.ylabel(f"Predicted U{i+1}")

  plt.title(
    f"Actual vs Predicted U{i+1}\n"
    f"R^2 = {R2:.4f}"
  )
  plt.legend()
  plt.grid(True)
  plt.axis('equal')

  plt.show()
```

If we run the code above, we would obtain the scatter plots shown in Figure 13. Based on the figure, we can see that our model is able to learn and approximate the mathematical relationship represented by Eq. (1), which itself is a typical matrix system of finite element analysis obtained from static force equilibrium. This allows our CNN+FFNN architecture to serve as a surrogate model for the FEM system.

From Figure 13, we can see that the data points are scattered quite close to the reference line, which represents where the predicted values are exactly equal to the actual values. This again highlights the good performance of the model. And again, there is a possibility that hyperparameter tuning could bring the data points closer or "tighter" to the reference line.

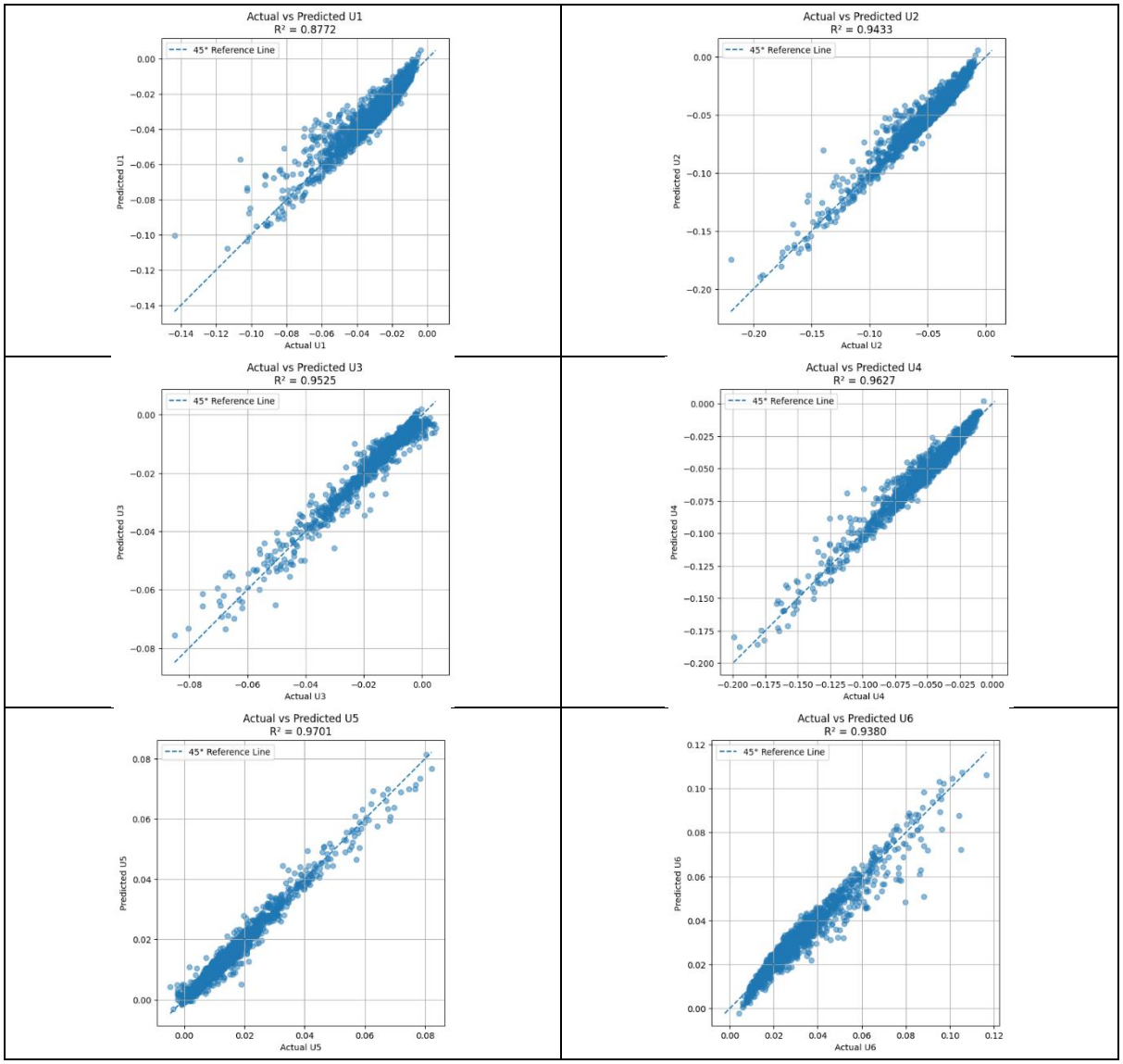


Figure 13: Scatter plots for the predictions of the DOFs of the FEM matrix system

References:

Bolandi, et.al. Bridging finite element and deep learning: High-resolution stress distribution prediction in structural components, Front. Struct. Civ. Eng. 2022, 16(11): 1365–1377

Celledoni, et.al. Predictions based on pixel data: Insights from PDEs and finite differences, Journal of Computational Physics 538 (2025) 114166

Herrmann, L., Kollmannsberger, S. Deep learning in computational mechanics: a review, Computational Mechanics (2024) 74:281–331

Zhong et.al. Structural Damage Features Extracted by Convolutional Neural Networks from Mode Shapes, Appl. Sci. **2020**, 10, 4247; doi:10.3390/app10124247

Zhang, et.al. A physics-informed convolutional neural network for the simulation and prediction of two-phase Darcy flows in heterogeneous porous media, Journal of Computational Physics 477 (2023) 111919